

HOSPITAL MANAGEMENT SYSTEM

Author:

- **Name:** Sonu Vishwakarma
 - **Roll Number:** 24f2006656
 - **Email ID:** 24f2006656@ds.study.iitm.ac.in
 - **About me:** I am a standalone student of IIT Madras and currently at diploma level. My short-term goal is to become a good data scientist and my hobby is always learning something new because flow is nature.
-

1. Project Overview

The **Hospital Management System** is a web-based application designed to digitize and streamline the administrative and clinical operations of a healthcare facility. The system addresses the need for efficient data management by replacing manual paper records with a centralized database.

The application serves three distinct user roles, each with specific permissions:

1. **Admin:** Manages doctors, patients, and views overall appointment logs.
2. **Doctor:** Manages their availability, views assigned patients, and updates medical history.
3. **Patient:** Register, see doctors by department, and book appointments.

2. Technology Stack & Justification

The project is built using a robust and lightweight technology stack suitable for rapid development and deployment.

Backend Framework: Flask (Python)

- **Why used:** Flask is a micro-web framework written in Python. It was chosen because it is lightweight, modular, and easy to set up. Unlike heavier frameworks (like Django), Flask allows for granular control over the application structure.
- **Role in Project:** It handles URL routing (e.g., `@app.route`), session management (for keeping users logged in), and connects the frontend HTML templates with the backend logic.

Database: SQLite

- **Why used:** SQLite is a C-language library that implements a small, fast, self-contained, high-reliability, full-featured, SQL database engine. It does not require a separate server process.
- **Role in Project:** It acts as the primary data store. Since the project targets rural connectivity or standalone usage, SQLite is ideal because the database is a single file (`hospital.db`) that requires no configuration.

Frontend: HTML & Jinja2 Templating

- **Why used:** Jinja2 is the default templating engine for Flask.
- **Role in Project:** It allows us to render dynamic data (like patient names or appointment slots) directly into HTML pages using Python-like syntax within the web pages.

Style: CSS

- **Why used:** Easy to understand and modify.
- **Role in Project:** Used for styling the application interface to ensure it is user-friendly and responsive.

3. Database Schema Design

The database is normalized to ensure data integrity and reduce redundancy. Below is the structure of the tables used in `hospital.db`.

1. **Users:** The central authentication table.
 - `id` (PK), `fullname`, `username`, `password`, `role`.
 - *Reason:* Centralizes login credentials to avoid duplicate auth logic for different roles.
2. **Doctors:** Stores professional details.
 - `id` (PK), `user_id` (FK to Users), `specialization`, `department`, `experience`.
 - *Constraint:* `ON DELETE CASCADE` ensures if a user is deleted, their doctor profile is too.
3. **Patients:** Stores personal health data.
 - `id` (PK), `user_id` (FK to Users), `age`, `gender`, `contact`, `blood_group`.
4. **Departments:** Stores hospital wing details.
 - `id` (PK), `name` (Unique), `overview`, `username` (Creator).
 - *Reason:* separating departments allows for better organization and easier updates to department descriptions.
5. **Appointments:** Manages bookings.
 - `id` (PK), `patient_id` (FK), `doctor_id` (FK), `date`, `time`, `status`.
6. **Availability:** Manages doctor schedules.

- `id` (PK), `doctor_id` (FK), `date`, `morning_slot`, `evening_slot`.
 - *Constraint:* `UNIQUE(doctor_id, date)` prevents a doctor from having duplicate schedule entries for the same day.
7. **Patient_History:** Permanent medical records.
- `id` (PK), `patient_id`, `doctor_id`, `diagnosis`, `prescription`, `medicines`.

4. Architecture and Features

Architecture: The project follows the **Model-View-Controller (MVC)** architectural pattern:

- **Model:** The `module.py` file handles all direct database interactions (SQL queries, connection management, table creation).
- **View:** The `templates/` folder contains HTML files that present data to the user.
- **Controller:** The `app.py` file contains the Flask routes. It acts as the bridge, receiving user input, calling the Model to fetch/save data, and returning the appropriate View.

Features Implemented:

1. **Role-Based Access Control (RBAC):** Distinct dashboards for Admins (system oversight), Doctors (patient management), and Patients (booking).
2. **Dynamic Availability System:** Doctors can set availability for specific dates. The logic uses `INSERT OR IGNORE` to handle schedule creation seamlessly.
3. **Conflict-Free Booking:** The system checks the `appointments` table for existing bookings at a specific `doctor_id`, `date`, and `time` before confirming a new slot, preventing double-booking.
4. **Medical History Archiving:** When an appointment is marked as "Completed," the data is automatically migrated to the `patient_history` table, preserving the medical record while clearing the active schedule.
5. **Department Search:** Patients can view doctor details filtered by department, with descriptions pulled dynamically from the database.

4. Entity-Relationship (ER) & Logic

key Relationships:

1. **Users to Doctors/Patients (1:1 Extension)**
 - **Logic:** Instead of storing login details in three separate tables, we use a single `users` table. The `doctors` and `patients` tables have a **One-to-One (1:1)** relationship with `users` via the `user_id` foreign key.
 - **Benefit:** Allows a single login screen (`/login`) to handle all user types seamlessly.

2. Doctors to Appointments (1:N)

- **Logic:** One doctor can have multiple appointments, but a specific appointment record belongs to only one doctor. This is a **One-to-Many (1:N)** relationship.

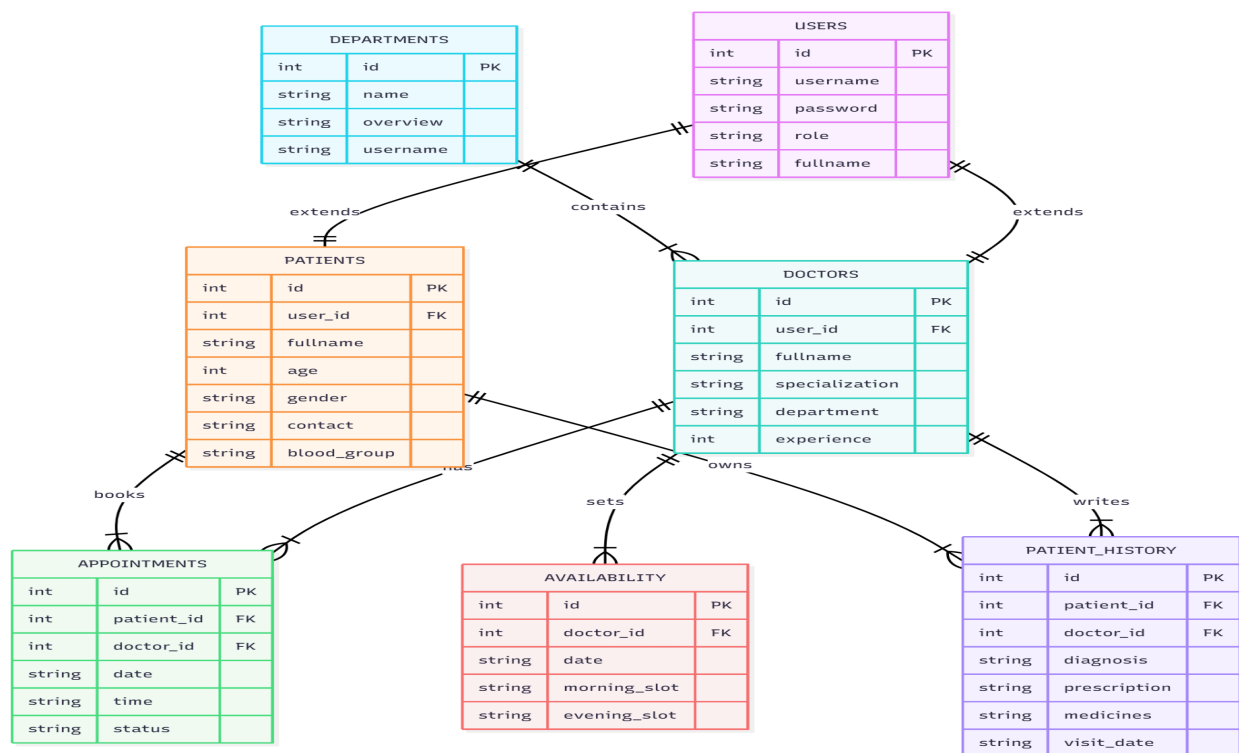
3. Patients to Appointments (1:N)

- **Logic:** Similarly, a single patient can book multiple appointments over time.

4. Doctors to Availability (1:N)

- **Logic:** A doctor has multiple availability entries (one per date). The constraint `UNIQUE(doctor_id, date)` ensures a doctor cannot have duplicate settings for the same day.

The system relies on Relational Database Management concepts.



5. API Design

While this application uses Server-Side Rendering (SSR), the routes function as internal API endpoints interacting with the database. The `api.yaml` file (submitted separately) details these endpoints. Key endpoints include:

- **POST /login:** Accepts `username`, `password`, and `role`. Returns a redirect to the specific dashboard (Admin/Doctor/Patient) upon success.
 - **POST /check_availability/<doctor_id>:** Accepts `date` and `time`. It checks the database for conflicts. If the slot is free, it writes to the `Appointments` table; otherwise, it returns an error.
 - **GET /admin:** Returns a JSON-like structure (rendered via HTML) of all doctors, patients, and upcoming appointments for the admin dashboard.
 - **POST /update_history/<appointment_id>:** Used by doctors to submit diagnosis data. This endpoint triggers a transaction that moves data from "Active Appointments" to "Patient History".
-

AI Usage Declaration

I, **Sonu Vishwakarma**, hereby declare that Artificial Intelligence (Chatgpt and Gemini) tools were utilized in the development. The specific usage was as follows:

1. **Code Optimization:** AI was used to refine the Python Flask logic, specifically for optimizing SQL queries and ensuring proper error handling (try-except blocks) in database connections.
 2. **Debugging:** AI assisted in identifying logic errors in the appointment slot booking mechanism and resolving Foreign Key constraint issues in SQLite.
 3. **CSS:** Use some parts in css like animation `@keyframe` etc.
-

Video:

Project Demo Link: [click here for video](#)

https://drive.google.com/file/d/19Hd2QAvL4NrZj5CglwDr_m8uQFfPzcrm/view?usp=drive_link